



Red Hat OpenShift AI Self-Managed 3.5

Building RAG applications with OGX

Building RAG applications with OGX in Red Hat OpenShift AI Self-Managed

Red Hat OpenShift AI Self-Managed 3.5 Building RAG applications with OGX

Building RAG applications with OGX in Red Hat OpenShift AI Self-Managed

Legal Notice

Copyright © Red Hat.

Except as otherwise noted below, the text of and illustrations in this documentation are licensed by Red Hat under the Creative Commons Attribution–Share Alike 3.0 Unported license . If you distribute this document or an adaptation of it, you must provide the URL for the original version.

Red Hat, as the licensor of this document, waives the right to enforce, and agrees not to assert, Section 4d of CC-BY-SA to the fullest extent permitted by applicable law.

Red Hat, the Red Hat logo, JBoss, Hibernate, and RHCE are trademarks or registered trademarks of Red Hat, LLC. or its subsidiaries in the United States and other countries.

Linux[®] is the registered trademark of Linus Torvalds in the United States and other countries.

XFS is a trademark or registered trademark of Hewlett Packard Enterprise Development LP or its subsidiaries in the United States and other countries.

The OpenStack[®] Word Mark and OpenStack logo are trademarks or registered trademarks of the Linux Foundation, used under license.

All other trademarks are the property of their respective owners.

Abstract

As an AI engineer or data scientist, you can build retrieval-augmented generation (RAG) applications with OGX in Red Hat OpenShift AI.

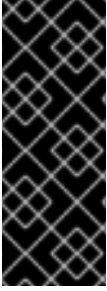
Table of Contents

PREFACE	3
CHAPTER 1. DEPLOYING A RAG STACK IN A PROJECT	4
1.1. OVERVIEW OF RAG	4
1.1.1. Audience for RAG	4
1.2. DEPLOYING A OGXSERVER INSTANCE	5
1.3. INGESTING CONTENT INTO A LLAMA MODEL	12
1.4. QUERYING INGESTED CONTENT IN A LLAMA MODEL	15
1.5. PREPARING DOCUMENTS WITH DOCLING FOR OGX RETRIEVAL	18
1.6. USING EXTERNAL S3-COMPATIBLE STORAGE FOR THE FILES API	21
1.6.1. External S3-compatible provider for the /v1/files endpoint	21
1.6.2. Creating secrets for the external S3-compatible files provider	22
1.6.3. Configuring the external S3-compatible provider for the /v1/files endpoint	23
1.6.4. Using the /v1/files endpoint with external S3-compatible storage	26
1.6.5. About the IAM policy for external S3-compatible files providers	29
1.6.6. Limitations of the external S3-compatible files provider	31
CHAPTER 2. EVALUATING RAG SYSTEMS WITH OGX	33
2.1. UNDERSTANDING RAG EVALUATION PROVIDERS	33
2.2. ABOUT OGX SEARCH TYPES	33
2.2.1. Supported search modes	33
2.3. USING RAGAS WITH OGX	35

PREFACE

Build retrieval-augmented generation (RAG) applications by using OGX in OpenShift AI.

CHAPTER 1. DEPLOYING A RAG STACK IN A PROJECT



IMPORTANT

This feature is currently available in Red Hat OpenShift AI 3.5 as a Technology Preview feature. Technology Preview features are not supported with Red Hat production service level agreements (SLAs) and might not be functionally complete. Red Hat does not recommend using them in production. These features provide early access to upcoming product features, enabling customers to test functionality and provide feedback during the development process.

As an AI engineer or data scientist, you can deploy a Retrieval-Augmented Generation (RAG) stack in a OpenShift AI project. The stack connects an OGX server to a vector store so that you can ingest, embed, and query domain-specific documents during inference.

Before you begin, ensure that a platform administrator has activated the OGX Operator, configured a vector store, and deployed an inference model in OpenShift AI.

To deploy the RAG stack in a project, complete the following tasks:

- Create a **OGXServer** instance to enable RAG functionality. This action connects OGX to the configured vector store and the inference model.
- Ingest domain data into the configured vector store by running Docling in an AI pipeline or Jupyter notebook. This process keeps the embeddings synchronized with the source data.
- Query the ingested content.

1.1. OVERVIEW OF RAG

Retrieval-augmented generation (RAG) in OpenShift AI enhances large language models (LLMs) by integrating domain-specific data sources directly into the model's context. Domain-specific data sources can be structured data, such as relational database tables, or unstructured data, such as PDF documents.

RAG indexes content and builds an embedding store that data scientists and AI engineers can query. When data scientists or AI engineers pose a question to a RAG chatbot, the RAG pipeline retrieves the most relevant pieces of data, passes them to the LLM as context, and generates a response that reflects both the prompt and the retrieved content.

By implementing RAG, data scientists and AI engineers can obtain tailored, accurate, and verifiable answers to complex queries based on their own datasets within a project.

1.1.1. Audience for RAG

The target audience for RAG is practitioners who build data-grounded conversational AI applications using OpenShift AI infrastructure.

For Data Scientists

Data scientists can use RAG to prototype and validate models that answer natural-language queries against data sources without managing low-level embedding pipelines or vector stores. They can focus on creating prompts and evaluating model outputs instead of building retrieval infrastructure.

For MLOps Engineers

MLOps engineers typically deploy and operate RAG pipelines in production. Within OpenShift AI, they manage LLM endpoints, monitor performance, and ensure that both retrieval and generation scale reliably. RAG decouples vector store maintenance from the serving layer, enabling MLOps engineers to apply CI/CD workflows to data ingestion and model deployment alike.

For Data Engineers

Data engineers build workflows to load data into storage that OpenShift AI indexes. They keep embeddings in sync with source systems, such as S3 buckets or relational tables to ensure that chatbot responses are accurate.

For AI Engineers

AI engineers architect RAG chatbots by defining prompt templates, retrieval methods, and fallback logic. They configure agents and add domain-specific tools, such as OpenShift job triggers, enabling rapid iteration.

1.2. DEPLOYING A OGXSERVER INSTANCE

You can deploy OGX with retrieval-augmented generation (RAG) by pairing it with a vLLM-served Llama 3.2 model. This module provides the following deployment examples of the OGXServer custom resource (CR):

- **Example A:** Remote Milvus (external service)
- **Example B:** Remote PostgreSQL with pgvector (external service, remote embeddings)

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- You have cluster administrator privileges for your OpenShift cluster.
- You are logged in to Red Hat OpenShift AI.
- You have activated the OGX Operator in OpenShift AI.
- You have deployed an inference model with vLLM (for example, **llama-3.2-3b-instruct**) and selected **Make deployed models available through an external route** and **Require token authentication** during model deployment. In addition, in **Add custom runtime arguments**, you have added **--enable-auto-tool-choice**.
- You have the correct inference model identifier, for example, **llama-3-2-3b**.
- You have the model endpoint URL ending with **/v1**, for example, **https://llama-32-3b-instruct-predictor:8443/v1**.
- You have the API token required to access the model endpoint.
- You have installed the PostgreSQL Operator version 14 or later and configured a PostgreSQL database for OGX metadata storage. For more information, see the documentation for "Deploying a OGX server".

- You have installed the OpenShift CLI (**oc**) as described in the appropriate documentation for your cluster:
 - [Installing the OpenShift CLI](#) for OpenShift Container Platform
 - [Installing the OpenShift CLI](#) for Red Hat OpenShift Service on AWS

Procedure

1. Open a new terminal window and log in to your OpenShift cluster from the CLI:
In the upper-right corner of the OpenShift web console, click your user name and select **Copy login command**. After you have logged in, click **Display token**. Copy the **Log in with this token** command and paste it in the OpenShift CLI (**oc**).

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

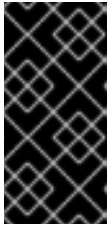
2. Create a secret that contains the inference model and the remote embeddings environment variables:

```
# Remote LLM
export INFERENCE_MODEL="llama-3-2-3b"
export VLLM_URL="https://llama-32-3b-instruct-predictor:8443/v1"
export VLLM_TLS_VERIFY="false" # Use "true" in production
export VLLM_API_TOKEN="<token identifier>"
export VLLM_MAX_TOKENS=16384

# Remote embedding configuration
export EMBEDDING_MODEL="nomic-embed-text-v1-5"
export EMBEDDING_PROVIDER_MODEL_ID="nomic-embed-text-v1-5"
export VLLM_EMBEDDING_URL="<embedding-endpoint>/v1"
export VLLM_EMBEDDING_API_TOKEN="<embedding-token>"
export VLLM_EMBEDDING_MAX_TOKENS=8192
export VLLM_EMBEDDING_TLS_VERIFY="true"

oc create secret generic ogx-secret -n <project-name> \
  --from-literal=INFERENCE_MODEL="$INFERENCE_MODEL" \
  --from-literal=VLLM_URL="$VLLM_URL" \
  --from-literal=VLLM_TLS_VERIFY="$VLLM_TLS_VERIFY" \
  --from-literal=VLLM_API_TOKEN="$VLLM_API_TOKEN" \
  --from-literal=VLLM_MAX_TOKENS="$VLLM_MAX_TOKENS" \
  --from-literal=EMBEDDING_MODEL="$EMBEDDING_MODEL" \
  --from-
literal=EMBEDDING_PROVIDER_MODEL_ID="$EMBEDDING_PROVIDER_MODEL_ID" \
  --from-literal=VLLM_EMBEDDING_URL="$VLLM_EMBEDDING_URL" \
  --from-literal=VLLM_EMBEDDING_TLS_VERIFY="$VLLM_EMBEDDING_TLS_VERIFY" \
  --from-literal=VLLM_EMBEDDING_API_TOKEN="$VLLM_EMBEDDING_API_TOKEN" \
  --from-
literal=VLLM_EMBEDDING_MAX_TOKENS="$VLLM_EMBEDDING_MAX_TOKENS"
```

3. Choose **one** of the following deployment examples:



IMPORTANT

In order to use OGX in a disconnected environment as well as enabling embeddings for deployments, you need to use the **remote::vllm** provider to set up a vLLM instance that uses an embedding model. For example, the **ibm-granite/granite-embedding-125m-english** model.

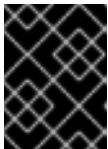
Example A: OGXServer with Remote Milvus

Use this example for production-grade or large datasets with an external Milvus service.

4. Create the Milvus connection secret:

```
# Required: gRPC endpoint on port 19530
export MILVUS_ENDPOINT="tcp://milvus-service:19530"
export MILVUS_TOKEN="<milvus-root-or-user-token>"
export MILVUS_CONSISTENCY_LEVEL="Bounded" # Optional; choose per your deployment

oc create secret generic milvus-secret \
  --from-literal=MILVUS_ENDPOINT="$MILVUS_ENDPOINT" \
  --from-literal=MILVUS_TOKEN="$MILVUS_TOKEN" \
  --from-literal=MILVUS_CONSISTENCY_LEVEL="$MILVUS_CONSISTENCY_LEVEL"
```



IMPORTANT

Use the **gRPC port 19530** for **MILVUS_ENDPOINT**. Ports such as **9091** are typically used for health checks and are not valid for client traffic.

5. In the OpenShift web console, select **Administrator** → **Quick Create** () → **Import YAML**, and create a CR similar to the following:

```
apiVersion: ogx.io/v1beta1
kind: OGXServer
metadata:
  name: ogx-upgrade-test
spec:
  distribution:
    name: rh-dev
  workload:
    replicas: 1
  overrides:
    env:
      - name: VLLM_URL
        valueFrom:
          secretKeyRef:
            key: VLLM_URL
            name: ogx-secret
      - name: VLLM_TLS_VERIFY
        valueFrom:
          secretKeyRef:
            key: VLLM_TLS_VERIFY
            name: ogx-secret
```

- name: VLLM_API_TOKEN
valueFrom:
secretKeyRef:
key: VLLM_API_TOKEN
name: ogx-secret
- name: EMBEDDING_MODEL
valueFrom:
secretKeyRef:
key: EMBEDDING_MODEL
name: ogx-secret
- name: VLLM_EMBEDDING_URL
valueFrom:
secretKeyRef:
key: VLLM_EMBEDDING_URL
name: ogx-secret
- name: VLLM_EMBEDDING_API_TOKEN
valueFrom:
secretKeyRef:
key: VLLM_EMBEDDING_API_TOKEN
name: ogx-secret
- name: VLLM_EMBEDDING_TLS_VERIFY
valueFrom:
secretKeyRef:
key: VLLM_EMBEDDING_TLS_VERIFY
name: ogx-secret
- name: EMBEDDING_PROVIDER_MODEL_ID
value: "<embedding-provider-model-id>"
- name: POSTGRES_HOST
value: "<postgres-host>"
- name: POSTGRES_PORT
value: "5432"
- name: POSTGRES_DB
value: "ogx_metadata"
- name: POSTGRES_USER
value: "ogx"
- name: POSTGRES_PASSWORD
valueFrom:
secretKeyRef:
key: password
name: <secret-name>
- name: ENABLE_PGVECTOR
value: "true"
- name: PGVECTOR_HOST
value: <postgres-host>
- name: PGVECTOR_PORT
value: "5432"
- name: PGVECTOR_DB
value: "pgvector"
- name: PGVECTOR_USER
valueFrom:
secretKeyRef:
key: pgvector-user
name: <secret-name>
- name: PGVECTOR_PASSWORD
valueFrom:
secretKeyRef:

```

    key: pgvector-password
    name: <secret-name>
storage:
  size: 5Gi

```



NOTE

The **rh-dev** value is an internal image reference. When you create the **OGXServer** custom resource, the OpenShift AI Operator automatically resolves **rh-dev** to the container image in the appropriate registry. This internal image reference allows the underlying image to update without requiring changes to your custom resource.

Example B: OGXServer with Remote PostgreSQL with pgvector

Use this example when you want to use a PostgreSQL database with the pgvector extension as the vector store backend. This configuration enables the pgvector provider and reads connection values from a secret. This example uses remote embeddings.

6. Create the pgvector connection secret:

```

export PGVECTOR_HOST="<pgvector-hostname>"
export PGVECTOR_PORT="5432"
export PGVECTOR_DB="<pgvector-database>"
export PGVECTOR_USER="<pgvector-username>"
export PGVECTOR_PASSWORD="<pgvector-password>"

oc create secret generic pgvector-connection -n <project-name> \
  --from-literal=PGVECTOR_HOST="$PGVECTOR_HOST" \
  --from-literal=PGVECTOR_PORT="$PGVECTOR_PORT" \
  --from-literal=PGVECTOR_DB="$PGVECTOR_DB" \
  --from-literal=PGVECTOR_USER="$PGVECTOR_USER" \
  --from-literal=PGVECTOR_PASSWORD="$PGVECTOR_PASSWORD"

```

7. In the OpenShift web console, select **Administrator** → **Quick Create** () → **Import YAML**, and create a custom resource similar to the following:

```

apiVersion: ogx.io/v1beta1
kind: OGXserver
metadata:
  name: lsd-llama-pgvector-remote
spec:
  replicas: 1
  server:
    containerSpec:
      resources:
        requests:
          cpu: "250m"
          memory: "500Mi"
        limits:
          cpu: 4
          memory: "12Gi"
  env:

```

PostgreSQL metadata store (required in {productname-short} 3.2)

- name: POSTGRES_HOST
value: <postgres-host>
- name: POSTGRES_PORT
value: "5432"
- name: POSTGRES_DB
value: <postgres-database>
- name: POSTGRES_USER
value: <postgres-username>
- name: POSTGRES_PASSWORD
valueFrom:
secretKeyRef:
name: <postgres-secret-name>
key: <postgres-password-key>

Remote LLM configuration

- name: INFERENCE_MODEL
valueFrom:
secretKeyRef:
name: ogx-secret
key: INFERENCE_MODEL
- name: VLLM_URL
valueFrom:
secretKeyRef:
name: ogx-secret
key: VLLM_URL
- name: VLLM_TLS_VERIFY
valueFrom:
secretKeyRef:
name: ogx-secret
key: VLLM_TLS_VERIFY
- name: VLLM_API_TOKEN
valueFrom:
secretKeyRef:
name: ogx-secret
key: VLLM_API_TOKEN
- name: VLLM_MAX_TOKENS
valueFrom:
secretKeyRef:
name: ogx-secret
key: VLLM_MAX_TOKENS

Remote embedding configuration

- name: EMBEDDING_MODEL
valueFrom:
secretKeyRef:
name: ogx-secret
key: EMBEDDING_MODEL
- name: EMBEDDING_PROVIDER_MODEL_ID
valueFrom:
secretKeyRef:
name: ogx-secret
key: EMBEDDING_PROVIDER_MODEL_ID
- name: VLLM_EMBEDDING_URL
valueFrom:
secretKeyRef:

```

    name: ogx-secret
    key: VLLM_EMBEDDING_URL
- name: VLLM_EMBEDDING_TLS_VERIFY
  valueFrom:
    secretKeyRef:
      name: ogx-secret
      key: VLLM_EMBEDDING_TLS_VERIFY
- name: VLLM_EMBEDDING_API_TOKEN
  valueFrom:
    secretKeyRef:
      name: ogx-secret
      key: VLLM_EMBEDDING_API_TOKEN
- name: VLLM_EMBEDDING_MAX_TOKENS
  valueFrom:
    secretKeyRef:
      name: ogx-secret
      key: VLLM_EMBEDDING_MAX_TOKENS

# Enable and configure pgvector provider
- name: ENABLE_PGVECTOR
  value: "true"
- name: PGVECTOR_HOST
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_HOST
- name: PGVECTOR_PORT
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_PORT
- name: PGVECTOR_DB
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_DB
- name: PGVECTOR_USER
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_USER
- name: PGVECTOR_PASSWORD
  valueFrom:
    secretKeyRef:
      name: pgvector-connection
      key: PGVECTOR_PASSWORD

- name: FMS_ORCHESTRATOR_URL
  value: "http://localhost"
name: ogx
port: 8321
distribution:
  name: rh-dev

```



NOTE

The **rh-dev** value is an internal image reference. When you create the **OGXServer** custom resource, the OpenShift AI Operator automatically resolves **rh-dev** to the container image in the appropriate registry. This internal image reference allows the underlying image to update without requiring changes to your custom resource.

8. Click **Create**.

Verification

- In the left-hand navigation, click **Workloads → Pods** and verify that the OGX pod is running in the correct namespace.
- To verify that the OGX Server is running, click the pod name and select the **Logs** tab. Look for output similar to the following:

```
INFO 2025-05-15 11:23:52,750 __main__:498 server: Listening on [':', '0.0.0.0']:8321
INFO: Started server process [1]
INFO: Waiting for application startup.
INFO 2025-05-15 11:23:52,765 __main__:151 server: Starting up
INFO: Application startup complete.
INFO: Uvicorn running on http://[':', '0.0.0.0']:8321 (Press CTRL+C to quit)
```

TIP

If you switch between vector store configurations, delete the existing pod to ensure the new environment variables and backing store are picked up cleanly.

1.3. INGESTING CONTENT INTO A LLAMA MODEL

You can quickly customize and prototype retrievable content by uploading a document and adding it to a vector store from inside a Jupyter notebook. This approach avoids building a separate ingestion pipeline. By using the OGX SDK, you can ingest documents into a vector store and enable retrieval-augmented generation (RAG) workflows.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have deployed a Llama 3.2 model with a vLLM model server.
- You have created a **OGXServer** instance.
- You have configured a PostgreSQL database for OGX metadata storage.
- You have configured an embedding model:
 - **Recommended:** You have configured a remote embedding model by using environment variables in the **OGXServer**.
- You have created a workbench within a project.

- You have opened a Jupyter notebook and it is running in your workbench environment.
- You have installed **ogx_client** version 1.0.0 or later in your workbench environment.
- You have installed **requests** in your workbench environment. This is required for downloading example documents.
- If you use a remote vector store or remote embedding model, your environment has network access to those services through OpenShift.

Procedure

1. In a new notebook cell, install the client:

```
%pip install ogx_client
```

2. Install the **requests** library if it is not already available:

```
%pip install requests
```

3. Import **OGXClient** and create a client instance:

```
from ogx_client import OGXClient
client = OGXClient(base_url="<ogx-base-url>")
```

4. List the available models:

```
models = client.models.list()
```

5. Verify that the list includes:

- At least one LLM model.
- At least one embedding model.

```
[Model(identifier='llama-32-3b-instruct', model_type='llm', provider_id='vllm-inference'),
 Model(identifier='nomic-embed-text-v1-5', model_type='embedding', metadata=
 {'embedding_dimension': 768})]
```

6. Select one LLM and one embedding model:

```
model_id = next(m.identifier for m in models if m.model_type == "llm")

embedding_model = next(m for m in models if m.model_type == "embedding")
embedding_model_id = embedding_model.identifier
embedding_dimension = int(embedding_model.metadata["embedding_dimension"])
```

7. (Optional) Create a vector store. Skip this step if you already have one.



NOTE

Provider IDs can differ between interfaces. In the Python SDK, you typically use the provider name directly (for example, **provider_id: "pgvector"**). In some CLI tools and examples, remote providers might use a prefixed identifier (for example, **--vector-db-provider-id remote-pgvector**). Use the provider ID format that matches the interface you are using.

Option 1: Remote Milvus (recommended for production)

```
vector_store = client.vector_stores.create(
    name="my_remote_milvus",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "milvus-remote",
    },
)
vector_store_id = vector_store.id
```

Ensure your **OGXServer** is configured with **MILVUS_ENDPOINT** and **MILVUS_TOKEN**.

Option 2: Remote PostgreSQL with pgvector

```
vector_store = client.vector_stores.create(
    name="my_pgvector_store",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "pgvector",
    },
)
vector_store_id = vector_store.id
```

Ensure that the pgvector provider is enabled in your **OGXServer** and that the PostgreSQL instance has the pgvector extension installed.

8. If you already have a vector store, set its identifier:

```
# vector_store_id = "<existing-vector-store-id>"
```

9. Download a PDF, upload it to OGX, and add it to your vector store:

```
import requests

pdf_url = "https://www.federalreserve.gov/aboutthefed/files/quarterly-report-20250822.pdf"
filename = "quarterly-report-20250822.pdf"

response = requests.get(pdf_url)
response.raise_for_status()

with open(filename, "wb") as f:
    f.write(response.content)
```

```

with open(filename, "rb") as f:
    file_info = client.files.create(
        file=(filename, f),
        purpose="assistants",
    )

vector_store_file = client.vector_stores.files.create(
    vector_store_id=vector_store_id,
    file_id=file_info.id,
    chunking_strategy={
        "type": "static",
        "static": {
            "max_chunk_size_tokens": 800,
            "chunk_overlap_tokens": 400,
        },
    },
)

print(vector_store_file)

```

Verification

- The call to **client.vector_stores.files.create()** succeeds and returns metadata for the ingested file.
- The vector store contains indexed chunks associated with the uploaded document.
- Subsequent RAG queries can retrieve content from the vector store.

1.4. QUERYING INGESTED CONTENT IN A LLAMA MODEL

You can use the OGX SDK in your Jupyter notebook to query ingested content by running retrieval-augmented generation (RAG) queries on content stored in your vector store. You can perform one-off lookups without setting up a separate retrieval service.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery Operator and NVIDIA GPU Operator. For more information, see [Installing the Node Feature Discovery Operator](#) and [Enabling NVIDIA GPUs](#).
- If you are using GPU acceleration, you have at least one NVIDIA GPU available.
- You have activated the OGX Operator in OpenShift AI.
- You have deployed an inference model, for example, the **llama-3.2-3b-instruct** model.
- You have created a **OGXServer** instance with:
 - PostgreSQL configured as the metadata store.
 - An embedding model configured, preferably as a remote embedding provider.
- You have created a workbench within a project and opened a running Jupyter notebook.

- You have installed **ogx_client** version 1.0.0 or later in your workbench environment.
- You have already ingested content into a vector store.



NOTE

This procedure requires that content has already been ingested into a vector store. If no content is available, RAG queries return empty or non-contextual responses.

Procedure

1. In a new notebook cell, install the client:

```
%pip install -q ogx_client
```

2. Import **OgxClient**:

```
from ogx_client import OgxClient
```

3. Create a client instance:

```
# Use the OGX service or route URL that is reachable from the workbench.
# Do not append /v1 when using ogx_client.
client = OGXClient(base_url="<ogx-base-url>")
```

4. List available models:

```
models = client.models.list()
```

5. Select an LLM. If you plan to register a new vector store, also capture an embedding model:

```
model_id = next(m.identifier for m in models if m.model_type == "llm")

embedding = next((m for m in models if m.model_type == "embedding"), None)
if embedding:
    embedding_model_id = embedding.identifier
    embedding_dimension = int(embedding.metadata.get("embedding_dimension", 768))
```

6. If you do not already have a vector store ID, register a vector store (choose one):

Option 1: Remote Milvus (recommended for production)

```
vector_store = client.vector_stores.create(
    name="my_remote_milvus",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "milvus-remote",
    },
)
vector_store_id = vector_store.id
```

Ensure your **OGXServer** sets **MILVUS_ENDPOINT** (gRPC port 19530) and **MILVUS_TOKEN**.

Option 2: Remote PostgreSQL with pgvector

```
vector_store = client.vector_stores.create(
    name="my_pgvector_store",
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "pgvector",
    },
)
vector_store_id = vector_store.id
```

Ensure the pgvector provider is enabled in your **OGXServer** and that the PostgreSQL instance has the pgvector extension installed. This option is suitable for production-grade RAG workloads that require durability and concurrency.

7. If you already have a vector store, set its identifier:

```
# vector_store_id = "<existing-vector-store-id>"
```

8. Query without using a vector store:

```
system_instructions = """You are a precise and reliable AI assistant.
Use retrieved context when it is available.
If nothing relevant is found, say so clearly."""

query = "How do you do great work?"

response = client.responses.create(
    model=model_id,
    input=query,
    instructions=system_instructions,
)

print(response.output_text)
```

9. Query by using the Responses API with file search:

```
response = client.responses.create(
    model=model_id,
    input=query,
    instructions=system_instructions,
    tools=[
        {
            "type": "file_search",
            "vector_store_ids": [vector_store_id],
        }
    ],
)

print(response.output_text)
```



NOTE

When you include the **file_search** tool with **vector_store_ids**, OGX retrieves relevant chunks from the specified vector store and provides them to the model as context for the response.

Verification

- The notebook returns a response without vector stores and a context-aware response when vector stores are enabled.
- No errors appear, confirming successful retrieval and model execution.

1.5. PREPARING DOCUMENTS WITH DOCLING FOR OGX RETRIEVAL

You can transform your source documents with a Docling-enabled pipeline and ingest the output into a OGX vector store by using the OGX SDK. This modular approach separates document preparation from ingestion while still enabling an end-to-end, retrieval-augmented generation (RAG) workflow.

The pipeline registers a vector store and downloads the source PDFs, then splits them for parallel processing and converts each batch to Markdown with Docling. It generates embeddings from the Markdown and stores them in the vector store, making the documents searchable through OGX.

Prerequisites

- You have installed OpenShift 4.19 or newer.
- You have enabled GPU support in OpenShift AI. This includes installing the Node Feature Discovery operator and NVIDIA GPU Operators. For more information, see [Installing the Node Feature Discovery operator](#) and [Enabling NVIDIA GPUs](#).
- You have logged in to the OpenShift web console.
- You have a project and access to pipelines in the OpenShift AI dashboard.
- You have created and configured a pipeline server within the project that contains your workbench.
- You have activated the OGX Operator in OpenShift AI.
- You have deployed an inference model, for example, the **llama-3.2-3b-instruct** model.
- You have configured a OGX deployment by creating a **OGXServer** instance to enable RAG functionality.
- You have created a workbench within a project.
- You have opened a Jupyter notebook and it is running in your workbench environment.
- You have installed the **ogx-client** version 0.3.1 or later in your workbench environment.
- You have installed local object storage buckets and created connections, as described in [Adding a connection to your project](#).
- You have compiled to YAML a pipeline that includes a Docling transform, either one of the RAG demo samples or your own custom pipeline.

- Your project quota allows between 500 millicores (0.5 CPU) and 4 CPU cores for the pipeline run.
- Your project quota allows from 2 GiB up to 6 GiB of RAM for the pipeline run.
- If you are using GPU acceleration, you have at least one NVIDIA GPU available.

Procedure

1. In a new notebook cell, install the client:

```
%pip install -q ogx-client
```

2. In a new notebook cell, import **OgxClient**:

```
from ogx_client import OgxClient
```

3. In a new notebook cell, assign your deployment endpoint to the **base_url** parameter to create a **OgxClient** instance:

```
client = OgxClient(base_url="http://<ogx-service>:8321")
```



NOTE

OgxClient requires the service root without the `/v1` path suffix. For example, use `http://ogx-service:8321`.

The `/v1` suffix is required only when you use OpenAI-compatible SDKs or send raw HTTP requests to the OpenAI-compatible API surface.

4. List the available models:

```
models = client.models.list()
```

5. Select the first LLM and the first embedding model:

```
model_id = next(m.identifier for m in models if m.model_type == "llm")
embedding_model = next(m for m in models if m.model_type == "embedding")
embedding_model_id = embedding_model.identifier
embedding_dimension = int(embedding_model.metadata.get("embedding_dimension", 768))
```

6. Register a vector store (choose one option). Skip this step if your pipeline registers the store automatically.

Remote Milvus

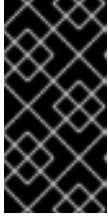
```
vector_store_name = "my_remote_db"
vector_store = client.vector_stores.create(
    name=vector_store_name,
    extra_body={
        "embedding_model": embedding_model_id,
        "embedding_dimension": embedding_dimension,
        "provider_id": "milvus-remote", # remote Milvus provider
    }
)
```

```

    },
)
vector_store_id = vector_store.id
print(f"Registered remote Milvus DB: {vector_store_id}")

```

Ensure your **OGXServer** includes **MILVUS_ENDPOINT** and **MILVUS_TOKEN** (gRPC :19530).



IMPORTANT

If you are using the sample Docling pipeline from the RAG demo repository, the pipeline registers the vector store automatically and you can skip the previous step. If you are using your own pipeline, you must register the vector store yourself.

7. In the OpenShift web console, import the YAML file containing your Docling pipeline into your project, as described in [Importing a pipeline](#).
8. Create a pipeline run to execute your Docling pipeline, as described in [Executing a pipeline run](#). The pipeline run inserts your PDF documents into the vector store. If you run the Docling pipeline from the [RAG demo samples repository](#), you can optionally customize the following parameters before starting the pipeline run:
 - **base_url**: The base URL to fetch PDF files from.
 - **pdf_filenames**: A comma-separated list of PDF filenames to download and convert.
 - **num_workers**: The number of parallel workers.
 - **vector_store_id**: The vector store identifier.
 - **service_url**: The Milvus service URL.
 - **embed_model_id**: The embedding model to use.
 - **max_tokens**: The maximum tokens for each chunk.
 - **use_gpu**: Enable or disable GPU acceleration.

Verification

1. In your Jupyter notebook, query the LLM with a question that relates to the ingested content:

```

system_instructions = """You are a precise and reliable AI assistant.
Use retrieved context when it is available.
If nothing relevant is found in the available files, say so clearly."""

prompt = "What can you tell me about the birth of word processing?"

# Query using the Responses API with file search
response = client.responses.create(
    model=model_id,
    input=prompt,
    instructions=system_instructions,
    tools=[
        {

```

```

        "type": "file_search",
        "vector_store_ids": [vector_store_id],
    }
],
)

print("Answer (with vector stores):")
print(response.output_text)

```

2. Query chunks from the vector store:

```

query_result = client.vector_io.query(
    vector_store_id=vector_store_id,
    query="word processing",
)
print(query_result)

```

- The pipeline run completes successfully in your project.
- Document embeddings are stored in the vector store and are available for retrieval.
- No errors or warnings appear in the pipeline logs or your notebook output.

1.6. USING EXTERNAL S3-COMPATIBLE STORAGE FOR THE FILES API

You can configure OpenShift AI to use an external S3-compatible object storage service as the backend for the OGX OpenAI-compatible **/v1/files** endpoint. This configuration enables file upload, storage, and retrieval for retrieval-augmented generation (RAG) and document-based workflows by using existing enterprise object storage infrastructure.

1.6.1. External S3-compatible provider for the /v1/files endpoint

The OGX Files API supports two providers in OpenShift AI: the default **inline::localfs** provider, which stores files on the local file system of the OGX pod, and the **remote::s3** provider, which stores file content in an external S3-compatible object storage service. Use the **remote::s3** provider when you require scalable, durable storage for file content that is independent of the OGX pod lifecycle, or when you must integrate with enterprise-managed storage platforms.

Support level: Developer Preview.

The **remote::s3** provider stores file content in an S3 bucket. File metadata, such as the file ID, filename, purpose, size, and timestamps, is stored in the PostgreSQL metadata store that OGX requires in OpenShift AI. The Files API metadata is managed automatically within the same PostgreSQL instance that holds metadata for the other OGX APIs. No additional database configuration is required.

The provider works with any object storage system that exposes an S3-compatible API, including the following examples:

- Amazon S3
- MinIO
- Ceph Object Gateway (RGW)
- Oracle Cloud Infrastructure (OCI) Object Storage, through the S3 Compatibility API

Since OGX interacts with the storage system through the S3 API, any storage technology is compatible as long as it implements the S3 API.

Benefits

Using external S3-compatible storage for the **/v1/files** endpoint provides the following advantages:

- Reuse existing, approved object storage services and governance controls for file content.
- Scale file content storage independently of OGX compute resources.
- Persist file content across OGX pod restarts and rescheduling events.
- Centralize file content across multiple AI applications and clusters.
- Meet compliance requirements by integrating with enterprise-managed storage platforms.

Compatibility and constraints

The **remote::s3** provider has the following compatibility characteristics and constraints:

- The provider is compatible with the OpenAI **/v1/files** API.
- The **inline::localfs** provider remains supported and requires no configuration changes for existing deployments.
- Migration from the **inline::localfs** provider to the **remote::s3** provider is not performed automatically. Files stored by one provider are not available through the other.
- The external storage system must implement a compatible S3 API. Provider-specific features that deviate from the S3 specification are not supported.
- The provider uses the same **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY** environment variables as the **remote::bedrock** inference provider. If you use both providers and require different credentials for each, use IAM roles for service accounts or a non-AWS S3-compatible backend.
- Some S3-compatible backends require additional client-side environment variables, such as checksum-handling settings. Consult the documentation for your S3-compatible backend for the configuration that it requires.

For information about the limitations of the **remote::s3** provider, see [Limitations of external S3-compatible files providers](#).

1.6.2. Creating secrets for the external S3-compatible files provider

To authenticate OGX to an external S3-compatible object storage service by using access keys, create a Kubernetes secret that contains the credentials.

If you intend to use IAM roles for service accounts (IRSA) or another short-lived credential mechanism, you can skip this procedure and configure the role binding for your OGX service account instead.

Prerequisites

- You have access to the project where your OGX resources are deployed.

- You have the access key ID and secret access key for your S3-compatible storage backend, or you have configured an IAM role for the OGX service account.

Procedure

1. Log in to your OpenShift cluster from the CLI:

```
$ oc login --token=<token> --server=<openshift_cluster_url>
```

2. Create a secret that contains the S3 credentials:

```
$ oc create secret generic s3-files-credentials \
  --from-literal=AWS_ACCESS_KEY_ID=<access_key_id> \
  --from-literal=AWS_SECRET_ACCESS_KEY=<secret_access_key> \
  -n <project>
```

In the previous command, replace **<access_key_id>** with the access key ID for your S3-compatible storage backend. Replace **<secret_access_key>** with the secret access key for your S3-compatible storage backend. Replace **<project>** with the name of the project where the **OGXServer** resource is deployed.

Verification

- The S3 credentials secret exists in the project. You can confirm this by running the following command:

```
$ oc get secret s3-files-credentials -n <project>
```

1.6.3. Configuring the external S3-compatible provider for the /v1/files endpoint

To use external S3-compatible object storage as the backend for the OGX **/v1/files** endpoint, update your **OGXServer** custom resource (CR) to enable the **remote::s3** provider and supply the required configuration through environment variables.

Prerequisites

- You have deployed a OGX server and configured a PostgreSQL database for OGX metadata storage. For more information, see [Deploying a OGXServer instance](#). Metadata for the **remote::s3** provider is stored in this same PostgreSQL instance automatically.
- You have created a secret that contains your S3 credentials. For more information, see [Creating secrets for the external S3-compatible files provider](#).
- If your S3 endpoint uses a TLS certificate signed by a private certificate authority (CA), you have configured the OGX server to trust that CA. For more information, see [Configuring a CA bundle for OGX](#).
- You have the S3 configuration details including: the external S3 endpoint URL, the bucket name, and the region value.
- The S3 bucket exists. Bucket creation is the responsibility of a storage administrator. For development environments where automatic bucket creation is acceptable, see the optional **S3_AUTO_CREATE_BUCKET** field in the configuration example.

- You have permission to edit custom resources in your project.

Procedure

1. Log in to the OpenShift AI web console as a cluster administrator.
2. From the **Project** list, select the project that contains your **OGXServer** CR.
3. Update your **OGXServer** CR to enable the **remote::s3** files provider and reference the secret that you created.
 - a. Click **Home** → **Search**.
 - b. From the **Resources** list, search for **OGXServer** and select it. The cluster also exposes a **OGXOperator** resource, which is an internal OpenShift AI resource that is managed by the Red Hat OpenShift AI Operator. Do not select **OGXOperator**.
 - c. From the list of **OGXServer** instances, click the name of the instance that you want to update.
 - d. Click the **YAML** tab.
 - e. Update the resource to include the following fields, and then click **Save**:

```
apiVersion: ogx.io/v1beta1
kind: OGXServer
metadata:
  name: my-ogx
  namespace: my-ogx-namespace
spec:
  server:
    containerSpec:
      env:
        - name: ENABLE_S3 ❶
          value: "true"
        - name: S3_BUCKET_NAME ❷
          value: "<bucket_name>"
        - name: AWS_DEFAULT_REGION ❸
          value: "<region>"
        - name: S3_ENDPOINT_URL ❹
          value: "<s3_endpoint_url>"
        - name: S3_AUTO_CREATE_BUCKET ❺
          value: "false"
        - name: AWS_ACCESS_KEY_ID ❻
          valueFrom:
            secretKeyRef:
              name: s3-files-credentials
              key: AWS_ACCESS_KEY_ID
        - name: AWS_SECRET_ACCESS_KEY
          valueFrom:
            secretKeyRef:
              name: s3-files-credentials
              key: AWS_SECRET_ACCESS_KEY
      name: ogx
```

```
port: 8321
distribution:
  name: rh-dev
```

- 1 Enables the **remote::s3** files provider.
- 2 Specifies the name of the S3 bucket where files are stored. S3 bucket names must be globally unique.
- 3 Specifies the region for the S3 bucket, for example, **us-east-1**. For non-AWS backends, set this value to match the region configuration of your backend, if it requires one. The default of **us-east-1** is appropriate for many S3-compatible backends, including MinIO.
- 4 Optional: Specifies the S3 endpoint URL for S3-compatible backends other than AWS S3. Omit this field when using AWS S3. Set this field for MinIO, Ceph Object Gateway, OCI Object Storage, or other S3-compatible backends.
- 5 Optional: When set to **"true"**, allows the provider to create the bucket if it does not exist. Requires the **s3:CreateBucket** IAM permission. Leave unset or set to **"false"** in production environments.
- 6 Specifies the S3 credentials, sourced from the secret that you created. Omit **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY** if the OGX service account uses an IAM role.



NOTE

For production deployments, Red Hat recommends authenticating to AWS S3 by using IAM roles for service accounts (IRSA) rather than static access keys. When you use an IAM role, omit the **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY** environment variables, and bind the IAM role to the service account used by the OGX pod.



NOTE

The **remote::s3** provider reads the same **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY** environment variables as the **remote::bedrock** inference provider. If you have configured both providers and require different credentials for each, use IAM roles for service accounts or a non-AWS S3-compatible backend.

Verification

- The OGX pod restarts and reaches the **Running** state.
- The pod logs show the resolved OGX configuration containing a Files provider entry with **provider_id: s3** and **provider_type: remote::s3**. You can also confirm provider registration by sending a **GET** request to the **/v1/providers** endpoint of the OGX API and verifying that an entry with **"api": "files"** and **"provider_id": "s3"** appears in the response.
- If the configured bucket does not exist and **S3_AUTO_CREATE_BUCKET** is set to **false**, the OGX pod enters **CrashLoopBackOff**. The pod logs include the following error: **RuntimeError: S3 bucket '<bucket_name>' does not exist. Either create the bucket manually or set**

'auto_create_bucket: true' in your configuration.

- File operations against the **/v1/files** endpoint complete successfully, confirming that OGX can communicate with the S3-compatible backend. For more information, see [Using the /v1/files endpoint with external S3-compatible storage](#).

1.6.4. Using the /v1/files endpoint with external S3-compatible storage

After you configure the **remote::s3** provider, you can manage files by using the OpenAI-compatible **/v1/files** endpoint. File content is stored in your S3 bucket, and file metadata is stored in the OGX PostgreSQL metadata store.

Prerequisites

- You have configured the **remote::s3** provider for the **/v1/files** endpoint. For more information, see [Configuring an external S3-compatible provider for the /v1/files endpoint](#).
- The OGX pod is running and the **remote::s3** provider has initialized successfully.
- You have the **curl** command-line tool installed in the environment from which you run the procedure.
- You have the service or route URL for the OGX API endpoint, without the **/v1** suffix.
- You have the API token required to access the OGX endpoint, if your deployment requires authentication.

Procedure

1. Open a terminal session in an environment that can reach the OGX API. The OGX service is reachable in the following ways:

In-cluster access

Run the commands from a pod in the same project as the OGX service, or from a workbench that has network access to the OGX service. The OGX Operator names the service **<distribution-name>-service**. Use that name with port 8321, for example:

```
OGX_URL="http://ogx-service:8321"
```

External access

Expose the OGX service by creating a route, and then run the commands from your local workstation. Use the route URL, for example:

```
OGX_URL="https://ogx-route.example.com"
```

For more information about the correct base URL format and how to find the URL for your deployment, see [OpenAI compatibility for RAG APIs in OGX](#).

2. Set the **OGX_URL** environment variable to the URL of your OGX service or route, without the **/v1** suffix. The example commands in this procedure append **/v1** as part of the endpoint path.
3. If your deployment requires authentication, set the **OGX_TOKEN** environment variable to your API token:

```
$ export OGX_TOKEN="<api_token>"
```

4. Upload a file. The **purpose** form field is required:

```
$ curl -X POST \
  "${OGX_URL}/v1/files" \
  -H "Authorization: Bearer ${OGX_TOKEN}" \
  -F purpose="assistants" \
  -F file="@<path_to_file>"
```

In the previous command, replace **<path_to_file>** with the local path to the file that you want to upload.

The output is similar to the following example:

```
{
  "object": "file",
  "id": "file-53b3fd75ca2c421c9a292ac63ff924ce",
  "bytes": 16,
  "created_at": 1778608944,
  "expires_at": null,
  "filename": "test.txt",
  "purpose": "assistants"
}
```

Note the value of the **id** field. You use this identifier in subsequent operations to refer to the file. The **expires_at** field is **null** for files that do not expire, which is the default behavior for files uploaded with **purpose="assistants"**.

5. List files:

```
$ curl -X GET \
  "${OGX_URL}/v1/files" \
  -H "Authorization: Bearer ${OGX_TOKEN}"
```

The output is similar to the following example:

```
{
  "data": [
    {
      "object": "file",
      "id": "file-53b3fd75ca2c421c9a292ac63ff924ce",
      "bytes": 16,
      "created_at": 1778608944,
      "expires_at": null,
      "filename": "test.txt",
      "purpose": "assistants"
    }
  ],
  "has_more": false,
  "first_id": "file-53b3fd75ca2c421c9a292ac63ff924ce",
  "last_id": "file-53b3fd75ca2c421c9a292ac63ff924ce",
  "object": "list"
}
```

When no files are present, the **data** array is empty and the **first_id** and **last_id** fields contain empty strings.

6. Retrieve file metadata:

```
$ curl -X GET \
  "${OGX_URL}/v1/files/<file_id>" \
  -H "Authorization: Bearer ${OGX_TOKEN}"
```

In the previous command, replace **<file_id>** with the identifier of the file, which is returned by the upload or list operation.

The output is similar to the following example:

```
{
  "object": "file",
  "id": "file-53b3fd75ca2c421c9a292ac63ff924ce",
  "bytes": 16,
  "created_at": 1778608944,
  "expires_at": null,
  "filename": "test.txt",
  "purpose": "assistants"
}
```

7. Retrieve file content:

```
$ curl -X GET \
  "${OGX_URL}/v1/files/<file_id>/content" \
  -H "Authorization: Bearer ${OGX_TOKEN}" \
  -o <output_path>
```

In the previous command, replace **<output_path>** with the local path where the downloaded file content is saved.

The endpoint returns the raw bytes of the file. The command does not produce console output when you use the **-o** option. Confirm that the file was downloaded successfully by checking that the output file exists and has the expected size:

```
$ ls -l <output_path>
```

The reported file size matches the **bytes** value returned by the metadata operation.

8. Delete a file:

```
$ curl -X DELETE \
  "${OGX_URL}/v1/files/<file_id>" \
  -H "Authorization: Bearer ${OGX_TOKEN}"
```

The output is similar to the following example:

```
{
  "id": "file-53b3fd75ca2c421c9a292ac63ff924ce",
  "object": "file",
}
```

```

    "deleted": true
  }

```

The **deleted** field is set to **true**, confirming that OGX has removed both the file metadata and the underlying S3 object.

Verification

- Uploaded files appear in the response from the list operation, with an entry in the **data** array whose **id** field matches the **id** returned by the upload operation, and whose **filename**, **bytes**, and **purpose** fields match the values that you supplied at upload time.
- Delete operations return a response with **"deleted": true**, and a subsequent list operation does not include the deleted file in the **data** array. The corresponding object is also removed from the underlying S3 bucket.
- The OGX pod logs record each operation as an HTTP access entry. For example, a successful upload appears in the logs as **"POST /v1/files HTTP/1.1" 200**.

Additional resources

- To make uploaded files available for retrieval-augmented generation (RAG) workflows, you must associate the file with a vector store by using the **/v1/vector_stores/{vector_store_id}/files** endpoint. For more information, see [Ingesting content into a Llama model](#).

1.6.5. About the IAM policy for external S3-compatible files providers

The **remote::s3** provider requires a minimum set of permissions on the S3 bucket that it uses. Grant only the permissions required for **/v1/files** operations, and avoid using credentials that provide access to multiple buckets or accounts. The following examples show least-privilege IAM policies that you can use as a starting point for AWS S3 deployments. Adapt the policies to your S3-compatible backend's access control mechanism as required.

Required permissions

The **remote::s3** provider requires the following permissions on the bucket:

```

{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket"
      ],
      "Resource": [
        "arn:aws:s3:::<bucket_name>",
        "arn:aws:s3:::<bucket_name>/*"
      ]
    }
  ]
}

```

```
}
]
}
```

In the previous policy, replace **<bucket_name>** with the name of the S3 bucket that the **remote::s3** provider uses.

Additional permission for automatic bucket creation

If **S3_AUTO_CREATE_BUCKET** is set to **true**, the provider also requires the **s3:CreateBucket** permission. Red Hat recommends pre-creating the bucket administratively rather than granting this additional permission to the workload.

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject",
        "s3:ListBucket",
        "s3:CreateBucket"
      ],
      "Resource": [
        "arn:aws:s3:::<bucket_name>",
        "arn:aws:s3:::<bucket_name>/*"
      ]
    }
  ]
}
```

Additional security guidance

When you configure the **remote::s3** provider, apply the following recommended security practices:

- Store access credentials in Kubernetes secrets, and restrict access to the project where the **OGXServer** resource is deployed.
- Use TLS to secure communication with the S3 endpoint. For S3 endpoints that use private CAs, configure the OGX server to trust the CA by using the operator's TLS configuration. For more information, see [Configuring a CA bundle for OGX](#).
- For production deployments, use IAM roles for service accounts (IRSA) instead of static access keys. IAM roles provide short-lived credentials and remove the need to store long-lived secrets in the cluster.
- Rotate access keys regularly when static credentials are used.
- Apply a bucket policy that restricts access to the specific principal that runs the OGX workload.
- OGX enforces access policies at the file metadata layer. File visibility and access through the **/v1/files** API is governed by the access policies that you configure on the OGX server, in addition to the IAM permissions and bucket policies that you configure on the S3 backend.

- The **remote::s3** provider does not enable server-side encryption on uploaded objects. If you require encryption at rest, configure default server-side encryption on the S3 bucket.

1.6.6. Limitations of the external S3-compatible files provider

The following limitations apply to the **remote::s3** provider for the OGX **/v1/files** endpoint in OpenShift AI.

File expiration

By default, uploaded files do not expire. To set a per-file expiration, specify the **expires_after** field at upload time. Files that are uploaded with the **batch** purpose expire 30 days after upload.

Server-side encryption

The provider does not enable server-side encryption on uploaded objects. If you require encryption at rest, configure default server-side encryption on the S3 bucket at the storage layer.

AWS session tokens

AWS session tokens are not supported. The provider accepts only long-lived access keys or IAM roles for service accounts (IRSA).

S3 key prefixes

The provider does not support organizing files under an S3 key prefix. All file objects are stored at the root of the bucket. To isolate files for different workloads, use separate buckets.

Multipart uploads

The provider does not support S3 multipart uploads. All files are uploaded as single objects. Very large files might fail to upload, take longer to upload than they would with multipart upload, or require additional pod memory.

Streaming downloads

The provider loads file content into memory before returning it to the client. Very large file downloads can require significant pod memory.

S3 addressing style

The provider uses the AWS SDK's default S3 addressing style. Some S3-compatible backends, such as on-premises Ceph deployments without virtual-host DNS configuration, require path-style addressing. For these backends, configure path-style addressing on the backend or in the network layer rather than on the provider.

Automatic bucket creation

By default, the provider expects the S3 bucket to exist before the provider starts. If the bucket does not exist and **S3_AUTO_CREATE_BUCKET** is not set to **true**, the OGX server logs an error that names the missing bucket. Red Hat recommends pre-creating the bucket administratively rather than enabling automatic bucket creation.

Shared AWS credentials

The **remote::s3** provider reads the same **AWS_ACCESS_KEY_ID** and **AWS_SECRET_ACCESS_KEY** environment variables as the **remote::bedrock** inference provider. If you use both providers and require different credentials for each, use IAM roles for service accounts or a non-AWS S3-compatible backend.

Additional resources

- [Activating the OGX Operator](#)
- [Select and deploy a vector database](#)

- [Deploying a Llama model with KServe](#)
- [Introduction to OpenShift AI](#)
- [Demystify RAG with OpenShift AI and Elasticsearch](#)
- [From Podman AI Lab to OpenShift AI](#)
- [Red Hat OpenShift AI learning](#)
- [Technology Preview Features Support Scope](#)

CHAPTER 2. EVALUATING RAG SYSTEMS WITH OGX

You can use the evaluation providers that OGX exposes to measure and improve the quality of your Retrieval-Augmented Generation (RAG) workloads in OpenShift AI. This section introduces RAG evaluation providers, describes how to use Ragas with OGX, and helps you choose the right provider for your use case.

2.1. UNDERSTANDING RAG EVALUATION PROVIDERS

OGX supports pluggable evaluation providers that measure the quality and performance of Retrieval-Augmented Generation (RAG) pipelines. Evaluation providers assess how accurately, faithfully, and relevantly the generated responses align with the retrieved context and the original user query. Each provider implements its own metrics and evaluation methodology. You can enable a specific provider through the configuration of the **OGXServer** custom resource.

OpenShift AI supports the following evaluation providers:

- **Ragas**: A lightweight, Python-based framework that evaluates factuality, contextual grounding, and response relevance.
- **AutoRAG**: Automatically optimize RAG configurations for your documents.
- **TrustyAI**: A Red Hat framework that evaluates explainability, fairness, and reliability of model outputs.

Evaluation providers operate independently of model serving and retrieval components. You can run evaluations asynchronously and aggregate results for quality tracking over time.

Additional resources

- [AutoRAG overview](#)

2.2. ABOUT OGX SEARCH TYPES

OGX supports keyword, vector, and hybrid search modes for retrieving context in retrieval-augmented generation (RAG) workloads. Each mode offers different tradeoffs in precision, recall, semantic depth, and computational cost.

2.2.1. Supported search modes

Keyword search Keyword search applies lexical matching techniques, such as TF-IDF or BM25, to locate documents that contain exact or near-exact query terms. This approach is effective when precise term-matching is required, such as searching for identifiers, names, or regulatory terms.

Keyword search example:

```
query_result = client.vector_io.query(
    vector_store_id=vector_store_id,
    query="FRBNY",
    params={
        "mode": "keyword",
        "max_chunks": 3,
        "score_threshold": 0.7,
    },
)
```

```
)  
  
print(query_result)
```

For more information about keyword-based retrieval, see [The Probabilistic Relevance Framework: BM25 and Beyond](#).

Vector search

Vector search encodes documents and queries as dense numerical vectors, known as embeddings, and measures similarity using metrics such as cosine similarity or inner product. This approach captures semantic meaning and supports contextual matching beyond exact word overlap.

Vector search example:

```
query_result = client.vector_io.query(  
    vector_store_id=vector_store_id,  
    query="FRBNY",  
    params={  
        "mode": "vector",  
        "max_chunks": 3,  
        "score_threshold": 0.7,  
    },  
)  
  
print(query_result)
```

For more information, see [Billion-scale similarity search with GPUs](#).

Hybrid search Hybrid search combines keyword and vector-based retrieval techniques, typically by blending lexical and semantic relevance scores. This approach aims to balance exact term matching with semantic similarity.

Hybrid search example:

```
query_result = client.vector_io.query(  
    vector_store_id=vector_store_id,  
    query="FRBNY",  
    params={  
        "mode": "hybrid",  
        "max_chunks": 3,  
        "score_threshold": 0.7,  
    },  
)  
  
print(query_result)
```

For more information, see [Sparse, Dense, and Hybrid Retrieval for Answer Ranking](#).

**NOTE**

Search mode availability depends on the selected vector store provider and its configured capabilities.

Not all providers support every search mode. For example, some providers might support vector search only, while keyword or hybrid search might be unavailable or return empty results. Always verify supported search modes for your chosen vector store provider.

2.3. USING RAGAS WITH OGX

You can use the Ragas (Retrieval-Augmented Generation Assessment) evaluation provider with OGX to measure the quality of your Retrieval-Augmented Generation (RAG) workflows in OpenShift AI. Ragas integrates with the OGX evaluation API to compute metrics such as faithfulness, answer relevancy, and context precision for your RAG workloads.

OGX exposes evaluation providers as part of its API surface. When you configure Ragas as a provider, the OGX server sends RAG inputs and outputs to Ragas and records the resulting metrics for later analysis.

Ragas evaluation with OGX in OpenShift AI supports the following deployment modes:

- **Inline provider** for development and small-scale experiments.
- **Remote provider** for production-scale evaluations that run as OpenShift AI AI pipelines.

You choose the mode that best fits your workflow:

- Use the inline provider when you want fast, low-overhead evaluation while you iterate on prompts, retrieval configuration, or model choices.
- Use the remote provider when you need to evaluate large datasets, integrate with CI/CD pipelines, or run repeated benchmarks at scale.

Additional resources

- [Demystify RAG with OpenShift AI and Elasticsearch](#)
- [From Podman AI Lab to OpenShift AI](#)
- [Red Hat OpenShift AI learning](#)